# WOW

Terminal del agente de fútbol

Prueba de trabajo · explicación del proyecto · octubre de 2026

QUÉ ES

**wow** es un espacio de trabajo para el agente (representante) de fútbolistas y su staff. Reúne en un solo sitio la cartera de jugadores con su valor y su contrato, el simulador de negociación, las valoraciones del staff, las noticias y los documentos, y lo expone además a asistentes de IA a través de un servidor MCP.

El proyecto está pensado como **producto multi-club**: cada club trabaja en su propio workspace, aislado del resto, y el acceso es **solo por invitación** (no cualquiera puede registrarse).

| Problema | Respuesta de wow |
|---|---|
| Los datos del jugador viven repartidos (hojas de calculo, chats, carpetas). | Una ficha por jugador con contrato, valor, contacto, rendimiento, noticias, documentos y opiniones del staff. |
| Se negocia sin saber cuánta fuerza se tiene. | *Palanca* (0-100) según los meses de contrato que quedan, y simulador con probabilidad de cierre y comisión esperada. |
| Decidir renovar o fichar es subjetivo. | Recomendaciones con reglas visibles y, por MCP, una matriz de decisión multicriterio; nunca se inventan datos. |
| Los agentes de IA no pueden usar los datos del club. | Servidor MCP con OAuth y permisos de solo lectura o edición elegidos por quien autoriza. |

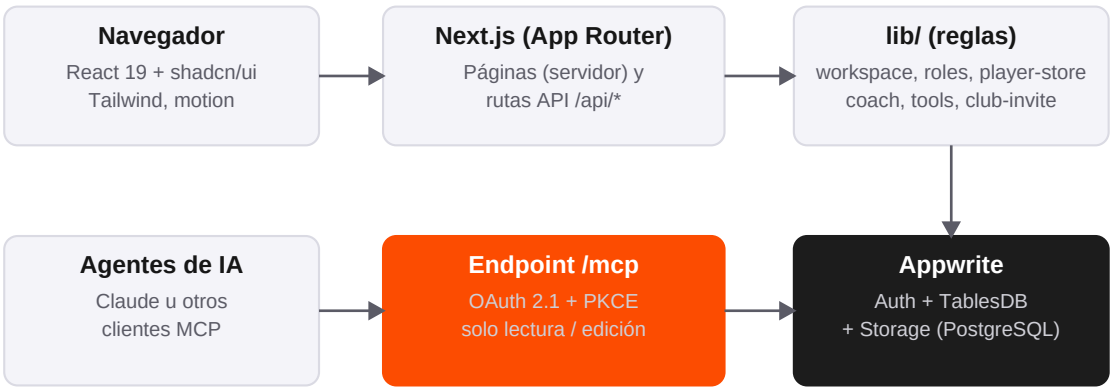
## Qué se puede hacer

| Sección | Qué ofrece |
|---|---|
| Resumen | Valor de la cartera, jugadores en zona de palanca, comisión esperada y siguiente revisión (contratos que vencen). |
| Jugadores | Listado y ficha: datos básicos, económicos, contacto y representante (editables), rendimiento, valoración técnica, noticias y clubes interesados. Alta y borrado con confirmación. |
| Negociaciones | Tablero por estado con la valoración de cada miembro del staff ("Mi valoración") y la del equipo. |
| Sala de decisión | Reúne las evidencias verificadas y la valoración del staff de un jugador antes de decidir. |
| Noticias | Noticias de cada jugador (Google News RSS). Se refrescan al abrir la bandeja si tienen más de una hora. |

| Sección | Qué ofrece |
|---|---|
| Comparador | Compara los perfiles de la cartera con los datos disponibles, marca lo que aún necesita una fuente y se exporta a Excel. |
| Carpetas | Escritorio estilo Finder: subcarpetas, arrastrar y soltar, cambiar nombre, acceso restringible por carpeta, vista previa de fotos, vídeo y audio (visor tipo galería con zoom) y subida de archivos de cualquier tipo hasta 1 GB por partes. |
| Workspace | Roles (propietario, administrador, editor, solo lectura), invitaciones por email, escudo del club como icono y quitar miembros. |

Las cuentas se crean por invitación: quien invita introduce un email y a esa persona le llega un correo con un enlace para poner su nombre y su contraseña.

# Arquitectura y tecnología




Toda lectura/escritura pasa por lib/ con el filtro de workspace y el rol del usuario; la API key de Appwrite solo existe en el servidor.

| Capa | Tecnología | Para qué |
|---|---|---|
| Interfaz | Next.js 16 (App Router), React 19, Tailwind CSS, shadcn/ui (Radix), motion, zustand | Páginas en servidor, componentes interactivos solo donde hacen falta. |
| Servidor | Rutas API de Next.js + lib/ (JavaScript) | Reglas de negocio únicas, compartidas entre la web y el MCP. |
| Datos | Appwrite (Auth, TablesDB sobre PostgreSQL, Storage) con node-appwrite | Usuarios, filas, archivos y migraciones versiónadas (db/migrations). |
| IA | Model Context Protocol (SDK oficial), OAuth 2.1 + PKCE | Herramientas para Claude u otros clientes. |
| Despliegue | Vercel (build normal); Appwrite Cloud o local con Docker | Las migraciones no corren en el build. |

# Seguridad y aislamiento entre clubes

Appwrite se usa desde el servidor con una API key de administrador, que **no aplica permisos por fila**. Por eso el aislamiento lo hace el código, con reglas pequeñas y probadas:

- **Workspace en toda consulta.** Cada lectura filtra por el workspace del usuario y cada fila leida por id se verifica antes de usarla; uno ajeno y uno inexistente responden igual (404).
- **Roles.** Las rutas que escriben comprueban el rol (editor o superior); gestionar miembros e icono exige administrador o propietario. Nadie puede quitarse a si mismo ni al propietario.
- **Acceso por carpeta.** Una carpeta puede restringirse a usuarios concretos; la restricción se hereda por las subcarpetas y borrar una rama exige acceso a toda ella.
- **Alta solo por invitación.** Iniciar sesión no crea ningun workspace. La invitación solo se activa con el email verificado: nadie entra registrandose con el email de otro.
- **Archivos.** Las subidas van firmadas (HMAC, caducan a las 6 h). Solo se muestran dentro de la página fotos, vídeo y audio, con *nosniff* y *CSP sandbox*; SVG y HTML nunca se ejecutan en el origen de la aplicación.
- **Tokens MCP.** Son JSON firmado con HMAC (sin estado); el permiso elegido va dentro del token y no se puede alterar. En cada llamada se vuelve a resolver la membresía del usuario.

# Alta de un club y onboarding

El flujo está pensado para que el equipo de wow controle quien entra y para que el usuario final lo tenga muy fácil:

| Paso | Qué ocurre |
|---|---|
| 1 | El club solicita acceso (email de su primer administrador y nombre del club). |
| 2 | Se da de alta con un comando: `bun run club:invite -- persona@club.com "Nombre del club"`. Se crea el workspace y se invita a esa persona como propietaria. |
| 3 | Le llega un email con un enlace. Pone su **nombre y una contraseña** en la página de onboarding y entra directamente a su workspace (también puede usar Google con ese email). |
| 4 | Desde "Compartir workspace" invita a su equipo; cada invitado recibe el mismo email. El enlace caduca a la hora y se puede pedir otro. |

# Servidor MCP para agentes de IA

El endpoint `/mcp` permite que un asistente como Claude trabaje con los datos de un workspace. Cada usuario autoriza con su cuenta y **solo ve su propio workspace**.
Al conectar, quien autoriza elige entre **solo lectura** (por defecto: consulta y análisis, sin herramientas de escritura) y **lectura y edición**.

| Tipo | Herramientas |
|---|---|
| Lectura | list_clients, portfolio_summary, valuation, negotiation_brief |
| Analíticas del cuerpo técnico | squad_alerts, squad_balance, lineup_suggestion, decision_matrix, player_report |
| Escritura (con permiso de edición) | add_player, update_player (rendimiento, valoración 0-100, valor, contacto), save_staff_opinión, add_suitor, remove_suitor, delete_player (exige confirm: true) |

- **Honestidad de los datos.** Las analíticas nunca estiman: lo que falta se excluye y se informa. La matriz de decisión no da nota a un jugador con menos del 50 % de los criterios con dato (sale como provisional y se ordena después).
- **Las mismas reglas que la web.** Las escrituras comparten código con la interfaz (rol, validación, filtro de workspace).

- **Pruebas.** Verificado de extremo a extremo con un token de solo lectura y otro de edición contra un servidor y un Appwrite locales.

# Calidad y forma de trabajo

- **470 tests** (bun test): fórmulas de negocio, aislamiento multi-tenant con dos workspaces, roles, subida por partes, OAuth y MCP.
- **Una sola puerta de calidad** (`bun run verify`) antes de subir cambios: lockfile, tests, build, *bun audit*, Snyk y React Doctor con **100/100 y 0 diagnósticos**. Corre igual en el hook pre-push, en el hook del agente de IA y en la CI.
- **Documentación viva** en el repositorio: README, arquitectura (mapa, multi-tenant, subida de archivos, MCP) y una guía de pruebas en navegador.
- **Cada cambio con su test** y commits pequenos con mensaje explicativo.

# Decisiones, límites y siguientes pasos

Lo que se decidió conscientemente y lo que queda por hacer:

- **Supuestos del simulador.** El 85 % del valor justo como mínimo del jugador, la comisión del 5 % y la pendiente logística son supuestos del modelo, no datos reales; están aislados en un solo archivo.
- **Tokens MCP sin revocación individual.** Al ser sin estado no se pueden revocar uno a uno (access 1 h, refresh 30 días); rotar la clave los invalida todos. Un siguiente paso sería una lista de revocación.
- **Subidas interrumpidas.** Si el navegador se cierra a mitad de una subida quedan partes huerfanas en Storage; falta una limpieza automática.
- **Historial de actividad.** Sería útil un registro de quien hizo que (ediciones y borrados, también por MCP).
- **Más herramientas MCP.** Búsqueda de jugadores, noticias y comentarios del staff, comparador y carpetas/archivos; y exponer los jugadores como recursos MCP.
- **Datos de rendimiento reales.** Conectar una fuente (minutos, goles, valor de mercado) para alimentar las analíticas sin teclearlos.

# Cómo ejecutarlo en local

```
bun install
bun run appwrite:up    # Appwrite en Docker
bun run appwrite:bootstrap  # proyecto y API key
bun run db:migrate
bun run dev               # http://localhost:3000
bun run club:invite -- persona@club.com "Mi club"
```

Requisitos: Bun, Docker y Node. El detalle (Google, SMTP, variables, producción) está en el README y en docs/.